

OOPS Concept

OOPs stands for Object-Oriented Programming system. It is a programming approach where code is organized using classes and objects.

- It helps to organized code, resue code, improve security, reduce duplication & make programs easier to maintain.

Real-Time example:

Imagine building a:

- School management system
- Banking App
- Hospital App
- E-Commerce Website

Thousands of data & Functions exist.

without OOPs:

everything becomes confusing.

With OOPs:

Code is separated into logical parts

Example :

- Student class
- Teacher class
- Course class

Each class handles its own work.

1. Class : A class is a blueprint/template for creating objects.

It defines:

- Variables(Attributes)
- Methods(Functions)
- Behavior of Object.

Why do we need a Class?

Without classes, we would have to write the same code repeatedly.

Purpose of a Class:

- Create a structure for objects
- Reuse code
- Represent real-world entities

Example:

```
class Student:
    pass
```

Here, Student is a class.

2. Object : An object is a real instance of class.

Why do we need an Object?

A class alone cannot perform any action. We need objects to:

- Store actual data
- Access class features
- Perform operations

Purpose of an Object

- Use class features
- Create real-world entities
- Store individual values

Example:

```
class Student:
    pass
s1 = Student()
```

Here:

- Student → Class
- s1 → Object

3.Constructor(__init__()): A Constructor is a special method that runs automatically whenever an object is created.

Syntax:

```
class Student:
    def __init__(self, name):
        self.name = name
```

Why do we use a Constructor?

Without a constructor, values must be assigned manually every time.

Constructor:

- Initializes objects automatically
- Saves coding effort
- Makes code cleaner

Example:

```
class Student:
    def __init__(self, name):
        self.name = name
s1 = Student("Ram")
print(s1.name)
```

Output: Ram

4. Self Keyword: self refers to the current object.

Why do we need self?

Suppose:

```
s1 = Student("Ram")
s2 = Student("Sam")
```

Python must know which object's data is being accessed.

self solves this problem.

Example:

```
class Student:
    def __init__(self, name):
        self.name = name
```

Here:

self.name (belongs to the current object.)

5. Encapsulation: Encapsulation is the process of wrapping data and methods together into a single unit (class) and restricting direct access to data.

Or

Encapsulation hides internal data and allows access only through specific method.

Why do we use Encapsulation?

- Protect data
- Improve security
- Control access to variables

Private Properties: Private variables are created using double underscore (__).

Example:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.__age = age
```

Accessing Private Variable Directly

```
p1 = Person("Ram",25)
print(p1.__age)
```

Output:AttributeError

Getter Method:Getter is used to access private data.

Example:

```
class Person:
    def __init__(self, name, age):
        self.__age = age
    def get_age(self):
        return self.__age
p1 = Person("Ram",25)
print(p1.get_age())
```

Output:25

Setter Method:Setter is used to modify private data.

Example:

```
class Person:
    def __init__(self, age):
        self.__age = age
    def set_age(self, age):
        if age > 0:
            self.__age = age
        else:
            print("Age must be positive")
    def get_age(self):
        return self.__age
p1 = Person(25)
p1.set_age(26)
print(p1.get_age())
```

output:26

Protected Properties: Protected variables use a single underscore (_).

Example:

```
class Employee:
    def __init__(self, salary):
        self._salary = salary
```

6. Inheritance: Inheritance is a mechanism in which one class acquires the properties and methods of another class.

Parent Class: The class being inherited from.

Also called:

- Base Class
- Super Class

Child Class: The class that inherits from another class.

Also called:

- Derived Class
- Sub Class

Simple Meaning

- Parent Class → Gives Features
- Child Class → Receives Features

Example:

```
class Animal:
    def sound(self):
        print("Animal Sound")
class Dog(Animal):
    pass
d = Dog()
d.sound()
```

Output: Animal Sound

7. Polymorphism: Polymorphism means "**Many Forms**".

The same method can perform different actions in different situations is called Polymorphism.

Types of Polymorphism

1. Compile-Time Polymorphism(Method Overloading)
2. Run-Time Polymorphism(Method Overriding)

Method Overriding: Method overriding occurs when a child class changes the behavior of a parent class method.

Or

When a child class provides a new implementation for a method that already exists in the parent class.

Example:

```
class Animal:
    def sound(self):
        print("Some Sound")
class Dog(Animal):
    def sound(self):
```

```
        print("Bark")
d = Dog()
d.sound()
```

Output: Bark

Method Overloading: Method overloading means having methods with the same name but different parameters.

Python does not support true method overloading directly, but it can be achieved using default arguments.

Example:

```
class Demo:
    def add(self, a=0, b=0):
        print(a + b)
d = Demo()
d.add(10)
d.add(10, 20)
```

Output:10

30

8. Abstraction: Abstraction is the process of hiding internal implementation details and showing only the essential features to the user.

Why do we use Abstraction?

- Reduce complexity
- Improve security
- Focus only on important features

Real-Life Example:

When you drive a car:

- You use the steering wheel, brake, and accelerator.
- You do not need to know how the engine works internally.

This is Abstraction.

Example:

```
from abc import ABC, abstractmethod
class Vehicle(ABC):
    @abstractmethod
    def start(self):
        pass
class Car(Vehicle):
    def start(self):
        print("Car Started")
c = Car()
c.start()
```

Output: Car Started

Dunder Methods in Python

Dunder methods are special methods in Python that begin and end with double underscores. They are used to define how objects behave with built-in operations like printing, comparison, addition, and length calculation.

- Dunder method or magic methods are special methods in python that begin & end with double underscore(__).
- The below code displays the magic methods inherited by int class.

Example : `print(dir(int))`

Output: `['__abs__', '__add__', '__and__', '__bool__', '__ceil__', '__class__', '.....', 'to_bytes']`

Explanation:

`dir(int)` lists all attributes and methods available in the int class, including magic methods such as: `__add__`, `__str__`, `__abs__`, `__bool__`.

You can list magic(dunder) methods for any python object via the terminal by following these steps:

1. Open terminal or command prompt and type `python3` to enter the python console.
2. Use the `dir(int)` function to list all methods
Eg: `>>>dir(int)`

Why are Dunder Methods Used?

They help customize object behavior such as:

- Object creation
- Printing objects
- Comparing objects
- Adding objects
- Finding length
- Iteration
- Operator overloading

Python Magic Methods:

Below are the lists of python magic methods and their uses.

1. Initialization and Construction Methods

1. `__new__()`: To get called when an object is created.

Example:

```
class Demo:
    def __new__(cls):
        print("Object is being created")
        return super().__new__(cls)
    def __init__(self):
        print("Constructor called")
d = Demo()
```

Output: Object is being created
Constructor called

2. `__init__()`: Constructor method called after object creation

Example:

```
class Student:
    def __init__(self, name):
        self.name = name
s = Student("Priya")
print(s.name)
```

Output: Priya

3. `__del__()`: Destructor method called when an object is deleted.

Example:

```
class Test:
    def __del__(self):
        print("Object Deleted")
t = Test()
del t
```

Output: Object Deleted

2. Numeric Magic Methods

1. `__trunc__()`: implements behaviour for `math.trunc()`.

Example:

```
import math
class Number:
    def __trunc__(self):
        return 10
n = Number()
print(math.trunc(n))
```

Output: 10

2. `__round__()`: Implements behaviour for the built-in `round()`.

Example:

```
class Number:
    def __round__(self, n=None):
        return 100
n = Number()
print(round(n))
```

Output: 100

3. `__abs__()`: Used with `abs()`.

Example:

```
class Number:
    def __abs__(self):
        return 50
n = Number()
print(abs(n))
```

Output: 50

3. `__neg__()`: Used for negative operator `(-)`.

Example:

```
class Number:
    def __neg__(self):
```

```
        return -100
n = Number()
print(-n)
```

Output:-100

5. `__pos__()`:Used for positive operator (+).

Example:

```
class Number:
    def __pos__(self):
        return 100
n = Number()
print(+n)
```

Output:100

3. Arithmetic Operator Methods

1. `__add__(self,other)`: implements behavior for the + operator(addition).

Example:

```
class Number:
    def __init__(self, value):
        self.value = value
    def __add__(self, other):
        return self.value + other.value
n1 = Number(10)
n2 = Number(20)
print(n1 + n2)
```

Output:30

2. `__sub__(self,other)`: implements behavior for the – operator(subtraction)

Example:

```
class Number:
    def __init__(self, value):
        self.value = value
    def __sub__(self, other):
        return self.value - other.value
n1 = Number(30)
n2 = Number(10)
print(n1 - n2)
```

Output:20

3. `__mul__(self,other)`: implements behavior for the * operator(multiplication).

Example:

```
class Number:
    def __init__(self, value):
        self.value = value
    def __mul__(self, other):
        return self.value * other.value
n1 = Number(5)
n2 = Number(4)
print(n1 * n2)
```

Output:20

4. `__truediv__(self,other)`: implements behavior for the / operator(true division).

Example:

```
class Number:
    def __init__(self, value):
        self.value = value
    def __truediv__(self, other):
        return self.value / other.value
n1 = Number(20)
n2 = Number(5)
print(n1 / n2)
```

Output: 4.0

5. `__mod__(self,other)`: implements behavior for the % operator(modulus).

Example:

```
class Number:
    def __init__(self, value):
        self.value = value
    def __mod__(self, other):
        return self.value % other.value
n1 = Number(10)
n2 = Number(3)
print(n1 % n2)
```

Output:1

4. String Magic Methods

1. `__str__(self)`: Defines behavior for when str() is called on an instances of your class.

Example:

```
class Car:
    def __str__(self):
        return "BMW Car"
c = Car()
print(c)
```

Output:BMW Car

2. `__repr__(self)`: To get called by built-in repr() method to return a machine readable representation of a type.

Example:

```
class Car:
    def __repr__(self):
        return 'Car("BMW")'
c = Car()
print(repr(c))
```

Output:Car("BMW")

3. `__hash__(self)`: It has to return an integer,and its result is used for quick key comparison in dictionaries.

Example:

```
class Demo:
    def __hash__(self):
        return 12345
d = Demo()
```

Output:12345

4. __bool__(self): Returns True or False to determine the truth value of an object,used in conditions and bool() calls.

Example:

```
class Demo:
    def __bool__(self):
        return False
d = Demo()
if d:
    print("True")
else:
    print("False")
```

Output:False

5. __dir__(self): This method to return a list of attributes of class.

Example:

```
class Demo:
    def __dir__(self):
        return ["name", "age"]
d = Demo()
print(dir(d))
```

Output:['age', 'name']

5. Comparison Magic Methods

1. __eq__(self,other): Defines behavior for the equality operator,==.

Example:

```
class Student:
    def __init__(self, marks):
        self.m = marks
    def __eq__(self, other):
        return self.m == other.m
s1 = Student(90)
s2 = Student(90)
print(s1 == s2)
```

Output:True

2. __ne__(self,other): Define behavior for the inequality operator,!=.

Example:

```
class Student:
    def __init__(self, marks):
        self.m = marks
    def __ne__(self, other):
        return self.m != other.m
s1 = Student(90)
s2 = Student(80)
```

```
print(s1 != s2)
```

Output:True

3. `__lt__(self,other)`: Define behavior for the less than operator,<.

Example:

```
class Number:
    def __init__(self, value):
        self.v = value
    def __lt__(self, other):
        return self.v < other.v
n1 = Number(5)
n2 = Number(10)
print(n1 < n2)
```

Output:True

4. `__gt__(self,other)`: Define behavior for the greater-than operator,>.

Example:

```
class Number:
    def __init__(self, value):
        self.v = value
    def __gt__(self, other):
        return self.v > other.v
n1 = Number(20)
n2 = Number(10)
print(n1 > n2)
```

Output:True

5. `__ge__(self,other)`: Define behavior for the greater than or equal to operator,>=.

Example:

```
class Number:
    def __init__(self, value):
        self.v = value
    def __ge__(self, other):
        return self.v >= other.v
n1 = Number(10)
n2 = Number(10)
print(n1 >= n2)
```

Output:True

6. Container Magic Methods

1. `__getitem__(self,index)`: These are special dunder(magic) method used to work with objects like lists or dictionaries using:[].

It is used to get/access a value using index or key.

- It is automatically called when using:object[index].

Example:

```
class Students:
    def __init__(self):
        self.names = ["Priya", "Rahul", "Kiran"]
    def __getitem__(self, index):
```

```
        return self.names[index]
s = Students()
print(s[0])
print(s[1])
```

Output: Priya
Rahul

2. **__setitem__()**: Used to update values using indexing or key.
It is automatically called when using object[index]=value.

Example:

```
class Students:
    def __init__(self):
        self.names = ["Priya", "Rahul", "Kiran"]
    def __setitem__(self, index, value):
        self.names[index] = value
s = Students()
s[1] = "Anu"
print(s.names)
```

Output: ['Priya', 'Anu', 'Kiran']

Combined Example:

```
class Numbers:
    def __init__(self):
        self.data = [10, 20, 30]
    def __getitem__(self, index):
        return self.data[index]
    def __setitem__(self, index, value):
        self.data[index] = value
n = Numbers()
print(n[1])
n[1] = 99
print(n.data)
```

Output: 20
[10, 99, 30]